

SPIRE/SPIFFE: Concepts, Configuration, and Infrastructure

What Is Running on This Cluster

Trust domain: `ipc.local`

SPIRE Server — StatefulSet (1 replica) on the cluster, NFS-backed SQLite for the CA keys and registration entries, listening on gRPC port 8081 via a headless service. Flux manages it from `manifests/spire/`.

SPIRE Agent — DaemonSet on all 6 nodes. `hostPID: true` (required for workload attestation). Exposes the Workload API at `/run/spire/sockets/agent.sock` as a hostPath volume. Each agent bootstraps trust using the `spire-bundle` ConfigMap, which the server populates with its CA cert via the `k8s_bundle` notifier.

SPIFFE and SPIRE — the Concepts

SPIFFE (Secure Production Identity Framework For Everyone) is a standard, not software. It defines:

- A URI format for workload identity: `spiffe://<trust-domain>/<path>`
- What a valid credential (SVID) looks like
- How an agent presents that credential to workloads (the Workload API)

SPIRE is the reference implementation of SPIFFE: a server that acts as a CA and a registration database, plus per-node agents that handle attestation and SVID issuance.

Node Attestation

The agent can't just claim "I'm a legitimate node" — the server needs proof. **Node attestation** is how the agent proves to the server that it's running on a real, authorized Kubernetes node.

This cluster uses `k8s_psat` (Kubernetes Projected Service Account Token):

1. Kubernetes issues a short-lived projected SAT to the agent pod (audience: `spire-server`, expiry: 7200s). This token is in the pod's filesystem at `/var/run/secrets/tokens/`.
2. The agent sends this token to the SPIRE server over gRPC.
3. The server calls the Kubernetes **TokenReview API** to validate it — Kubernetes confirms the token is real, which SA issued it, and which pod it was issued to.
4. The server checks its allow-list (`spire:spire-agent` SA) and grants the agent a **Node SVID** with SPIFFE ID `spiffe://ipc.local/k8s-node`.

From that point on, the agent authenticates to the server using the Node SVID (mTLS on the gRPC channel). The PSAT is only used once for initial bootstrapping.

Why PSAT instead of something else? These nodes have no TPM, no cloud provider identity (AWS EC2 instance identity documents, GCP GCE tokens, etc.), and no pre-provisioned join token — PSAT is the right choice for bare-metal Kubernetes. The trust root is "Kubernetes itself validated this token."

Workload Attestation

After a node is attested, the agent needs to verify that a calling workload is what it claims to be before issuing an SVID. This happens entirely on-node; no server round-trip is needed.

The flow for this cluster (k8s workload attestor):

1. Workload connects to `/run/spire/sockets/agent.sock` (a Unix domain socket).
2. The agent reads **SO_PEERCRE**D from the socket — this gives the calling process’s PID. This is why `hostPID: true` is required; without it, PIDs are namespaced and **SO_PEERCRE**D returns 0.
3. The agent reads `/proc/<pid>/cgroup` to extract the container ID.
4. The agent calls the **kubelet API** (using `MY_NODE_NAME`) to get pod metadata for that container: namespace, service account, pod name, labels, etc.
5. The agent compares that metadata against its **registration entries** (syncd from the server).
6. If a match is found, the agent issues an SVID for the matching SPIFFE ID.

`use_new_container_locator: true` in the agent config causes the agent to try `mountinfo` first (more reliable with `cgroupv2`) before falling back to the `cgroup` path.

Pelagos Compatibility Fixes

Two Pelagos bugs affected SPIRE workload attestation on this cluster:

Bug	Symptom	Fixed
<code>hostPID: true</code> ignored (#299)	SO_PEERCRE D returned PID 0; server rejected	v0.65.7
32-char container IDs (#301)	SPIRE regex expected 64-char hex; attestation failed	v0.65.8

Both are fixed in the running version (v0.65.47).

Registration Entries

Registration entries are the policy database — they tell the server (and thus agents) which workloads should receive which SPIFFE IDs. Each entry has:

- **SPIFFE ID**: the identity to issue (`spiffe://ipc.local/demo-app`)
- **Parent ID**: who must attest the caller first (`spiffe://ipc.local/k8s-node` — meaning “the workload must be on an attested node”)
- **Selectors**: the conditions that must all match (`k8s:ns:spire-demo, k8s:sa:demo-sa`)

Current Pattern: Imperative Registration

A Kubernetes Job runs `kubectl exec` into the server pod and calls `spire-server entry create`. This works but has a GitOps gap — entries live in SQLite on the NFS PVC, not in Git. If the PVC is lost or the cluster is rebuilt, entries must be re-registered.

GitOps Alternative: ClusterSPIFFEID

The **SPIRE Controller Manager** provides a **ClusterSPIFFEID** CRD. Entries are declared as Kubernetes resources in Git; the controller reconciles them into SPIRE continuously. This closes the GitOps gap. Architected in `docs/spire-trust-chain-clusterspiffeid.mmd` but not yet deployed.

SVIDs — the Two Forms

SVID = SPIFFE Verifiable Identity Document. Two formats:

X.509 SVID

A standard X.509 certificate with one key addition: the SPIFFE ID is encoded in the certificate's **Subject Alternative Name (SAN)** as a URI (`spiffe://ipc.local/demo-app`). Inspect with:

```
openssl x509 -in svid.pem -text -noout | grep URI
```

Properties:

- **Short-lived** (default TTL: 1 hour). The agent rotates them automatically before expiry, typically at the 50% mark.
- Carries the full chain: leaf cert + intermediate(s) up to the trust bundle.
- Used for **mTLS** — both sides present their SVID as the TLS client/server cert, and verify the peer's cert against the trust bundle. This is what experiment 21 demonstrates.
- Opaque to network intermediaries (the identity is in the cert, not the payload).

JWT SVID

A signed JWT with the SPIFFE ID in the `sub` claim and an `aud` (audience) claim.

Properties:

- Also short-lived. **Not** automatically rotated — the workload must re-fetch when it needs a fresh one.
- Used when mTLS is not possible: HTTP APIs where identity must be passed in a header (`Authorization: Bearer <jwt>`), or when a network intermediary (load balancer, API gateway) terminates TLS before reaching the workload.
- The receiving service verifies the JWT signature against the trust bundle's public key, not by doing a TLS handshake.
- Fetch with: `spire-agent api fetch jwt -audience <target-service> -socketPath ...`
- The **audience field matters**: a JWT fetched for audience `api-gateway` won't be accepted by a verifier expecting `payments-service`. This prevents token replay across services.

When to Use Which

Scenario	Format
Service-to-service where you control both ends	X.509 (mTLS)
HTTP API, identity passed in header	JWT

Scenario	Format
Identity through a TLS-terminating proxy	JWT
Authenticating to Vault (JWT auth method)	JWT
Envoy/service mesh transparent mTLS	X.509

The Trust Bundle

The trust bundle is the SPIRE CA's root certificate(s). Everything signed by SPIRE is verifiable against it. On this cluster:

- The server writes it to the `spire-bundle` ConfigMap in the `spire` namespace (via the `k8s_bundle` notifier).
- Agents mount it at startup to bootstrap their TLS connection to the server.
- Workloads receive it alongside their SVID when calling the Workload API — so they can verify peers without any pre-shared secrets or manual cert distribution.

Trust bundle rotation: If the server's CA rotates (either on schedule or forced), it publishes the new root alongside the old one during a transition window. SVIDs issued under the old CA remain valid until expiry; verifiers must accept both roots during the overlap. This is automatic in SPIRE.

Experiments Running on This Cluster

Experiment 11: Workload SVID Fetch

Namespace `spire-demo`, service account `demo-sa`. A demo pod fetches its X.509 SVID via the Workload API and prints the SPIFFE ID and cert details using `openssl`. Proves end-to-end attestation works.

Registration entries created by an imperative Job:

SPIFFE ID	Parent	Selectors
<code>spiffe://ipc.local/k8s-node</code>	<code>spiffe://ipc.local/spire-server</code>	<code>k8s:server:cluster:ipc</code> (node entry)
<code>spiffe://ipc.local/demo-app</code>	<code>spiffe://ipc.local/k8s-node</code>	<code>k8s:demo:spire-demo, k8s:sa:demo-sa</code>

Experiment 21: mTLS Between Two Workloads

Namespace `mtls-demo`. An `mtls-server` and `mtls-client` each receive unique X.509 SVIDs from SPIRE. The server runs `openssl s_server` requiring client cert (`-Verify 1`); the client runs `openssl s_client` presenting its SVID. Both sides verify against the SPIRE trust bundle. Zero pre-shared secrets.

SPIFFE ID	Workload
<code>spiffe://ipc.local/mtls-server</code>	<code>mtls-server</code> Deployment (<code>mtls-server-sa</code>)
<code>spiffe://ipc.local/mtls-client</code>	<code>mtls-client</code> Pod (<code>mtls-client-sa</code>)

TPM Hardware on This Cluster

All six nodes have TPM 2.0 chips with manufacturer-provisioned Endorsement Key (EK) certificates stored in TPM NV (non-volatile) memory. The EK cert is the hardware root of trust: it is signed by the manufacturer's CA at the factory and proves that the EK private key is bound to a specific, real TPM chip. The EK private key never leaves the chip.

EK Certificate Inventory

Node	Chip	Manufacturer	EK Cert Issuer	Valid Until
ipc4	Nuvoton NPCT75x	Nuvoton (TW)	NPCTxxx ECC384 LeafCA 022111	2043
ipc5	Nuvoton NPCT75x	Nuvoton (TW)	NPCTxxx ECC384 LeafCA 022111	2043
ipc6	Nuvoton NPCT75x	Nuvoton (TW)	NuvotonTPMRootCA2210	2042
ipc7	Nuvoton NPCT75x	Nuvoton (TW)	NuvotonTPMRootCA2211	2042
ipc8	Infineon SLB9672	Infineon (DE)	OPTIGA(TM) TPM 2.0 RSA CA 061	2037
ipc9	Infineon SLB9672	Infineon (DE)	OPTIGA(TM) TPM 2.0 RSA CA 061	2037

`tpm2-tools` is installed on all nodes (`apt install tpm2-tools`). The EK cert can be read with `sudo tpm2_getekcertificate -o /tmp/ek.crt`.

Are Nuvoton and Infineon Equivalent?

For SPIRE attestation: yes, completely. The TPM 2.0 attestation protocol is standardized by the TCG (Trusted Computing Group) and is identical regardless of manufacturer. SPIRE validates the EK cert chain and does a challenge-response over the EK private key; the manufacturer is irrelevant to the protocol.

In general terms, Infineon OPTIGA is the dominant TPM manufacturer globally and is used in FIPS-certified and government-cleared systems. Nuvoton NPCT is a legitimate TCG-certified TPM 2.0 part commonly shipped by HP in their business machines. Neither has active unpatched vulnerabilities. The historical ROCA vulnerability (CVE-2017-15361) affected Infineon RSA key *generation* in old firmware; ipc8/9 run firmware 15.0.0.22, well past the patched version.

ipc8/9 have both RSA and ECC EK certs (Infineon provisions both; Nuvoton nodes only have one). ECC P-384 is more modern than RSA 2048 — a mild theoretical advantage, not meaningful in practice.

ipc6 and ipc7 have different Nuvoton root CAs (2210 vs 2211) despite being the same hardware model. This reflects different manufacturing batches. For SPIRE attestor configuration you will need to trust all four manufacturer CA chains: two Nuvoton, one Infineon RSA, one Infineon ECC.

What Happens After 2037 (ipc8/9 EK Cert Expiry)?

You do not replace the TPM or the hardware. The TPM chip is permanent — it's soldered to the motherboard. What expires in 2037 is the manufacturer-signed *certificate* for the EK public key, not the EK key itself. The hardware continues to function normally.

What changes is your attestation policy. Currently, full EK cert chain validation works like this: “I trust this node because its EK cert chains to Infineon’s CA, which means Infineon vouches that this is a real TPM.” After expiry, that chain is stale.

The alternative — which is always available and is what most production deployments use alongside cert-chain validation — is **EK public key fingerprinting**: during initial enrollment, you record the EK public key of each node. The attestor then checks: “I trust this node because it can prove possession of the private key corresponding to *this enrolled public key*.” There is no expiry on a public key fingerprint. You lose the manufacturer-provenance guarantee (that the key is in a real TPM) but retain the cryptographic identity guarantee (that this is the same hardware you enrolled).

In practice for this cluster: re-enrollment in 2037 is the right move — you run through node attestation setup once more at that point and record the EK fingerprints. Or you pre-enroll fingerprints now alongside cert chain validation, and the transition in 2037 is a config change rather than a re-enrollment.

TPM Attestation: Plugin Choice and Implementation Plan

Plugin: `tpm_devid` (built-in to SPIRE)

SPIRE 1.9.6 has a native TPM node attestor called `tpm_devid`, stable since SPIRE 1.0.1 (2021). No external plugins are needed. The Bloomberg community plugin (`bloomberg/spire-tpm-plugin`) is archived and unmaintained — ignore it.

How `tpm_devid` Attestation Works

The attestation is two-part, which is what makes it stronger than a simple hardware value check:

1. **Proof of key possession** — the agent signs a server-issued nonce with its DevID private key. The server verifies the signature against the DevID certificate’s public key.
2. **Proof of TPM residency** — the server performs a **credential activation challenge** using the node’s EK public key. It encrypts a secret to the EK such that only the TPM holding the EK can decrypt it, and only if the DevID key is also loaded in that same TPM. The agent returns the decrypted secret, proving both keys reside in the same physical chip.

This two-part proof means an attacker can’t just copy key material off a node — possession of the DevID private key blob is not enough; it must be activated inside the TPM that holds the EK.

The SPIFFE ID issued by the `tpm_devid` attestor takes the form: `spiffe://ipc.local/spire/agent/tpm_devid/<`

What tpm_devid Requires

Per node (out-of-band provisioning): - A **DevID key pair generated inside the TPM** — the private key never leaves the chip; it exists only as an encrypted TPM key blob (`devid.priv`) paired with a public key blob (`devid.pub`) - A **DevID certificate** signed by a CA you control (the DevID CA), containing the public key

SPIRE Server: - `devid_ca_path` — the DevID CA cert (validates the DevID cert chain on each agent) - `endorsement_ca_path` — all manufacturer EK CA certs (validates the EK cert chain, completing the hardware root-of-trust verification)

This cluster requires four EK CA certs: Nuvoton NPCTxxx LeafCA 022111, NuvotonTPMRootCA2210, NuvotonTPMRootCA2211, and Infineon OPTIGA TPM 2.0 RSA CA 061.

SPIRE Agent: - `devid_cert_path` — path to the signed DevID certificate (PEM) - `devid_priv_path` — path to the TPM private key blob - `devid_pub_path` — path to the TPM public key blob

Migration Impact

Switching from `k8s_psat` to `tpm_devid` invalidates all existing agent registrations — the Node SVID format changes. The SPIRE server's registration database must be cleared and the registration jobs re-run after the switch. Since entries are currently managed by imperative Jobs, this is re-running those jobs, not a data migration.

TPM Attestation: Phased Implementation Plan

Phase 1 — DevID CA

Create an offline CA used exclusively for signing DevID certificates. This CA's cert becomes the `devid_ca_path` on the SPIRE server. It does not need to be online after provisioning; it only signs certs during node provisioning or reprovisioning.

Steps: - Generate CA key and self-signed cert (openssl, RSA 4096 or EC P-384) - Store the CA key offline (not on the cluster) - Store the CA cert in `config/tpm/devid-ca.pem` (committed to Git — public material only)

Phase 2 — Manufacturer EK CA Bundle

Fetch the four manufacturer root/intermediate CA certs from their public PKI URLs and assemble them into a single bundle file (`config/tpm/endorsement-ca-bundle.pem`).

Node(s)	CA to fetch	URL
ipc4, ipc5	Nuvoton NPCTxxx ECC384 LeafCA 022111	Nuvoton PKI
ipc6	NuvotonTPMRootCA2210	Nuvoton PKI
ipc7	NuvotonTPMRootCA2211	Nuvoton PKI
ipc8, ipc9	Infineon OPTIGA TPM 2.0 RSA CA 061	https://pki.infineon.com/

The Infineon CA URL is embedded in the EK cert's AIA extension (already visible in the EK cert output). Nuvoton's equivalent must be looked up from their PKI portal.

Phase 3 — Node Provisioning Script

Write `scripts/provision-tpm-devid.sh` that runs on a single node and produces the three files the SPIRE agent needs. The script:

1. Creates a TPM primary key under the Endorsement hierarchy: `tpm2_createprimary -C e -g sha256 -G rsa -c /tmp/primary.ctx`
2. Generates the DevID key under it: `tpm2_create -C /tmp/primary.ctx -g sha256 -G rsa -r /etc/spire/devid.priv -u /etc/spire/devid.pub`
3. Loads the key to get a handle and extracts the public key for the CSR: `tpm2_load, tpm2_readpublic`
4. Produces a CSR (using openssl with the TPM public key)
5. Returns the CSR — signing happens off-node with the DevID CA key
6. Writes the signed cert to `/etc/spire/devid.crt`

The script is idempotent: if blobs already exist and the cert is still valid, it exits cleanly. Re-provisioning (after node reinstall) regenerates everything.

The DevID CA signing step is intentionally separate from the provisioning script — the CA key is offline and signing is a deliberate human action, not automated.

Phase 4 — SPIRE Server Config Update

Update `manifests/spire/server-config.yaml`: - Replace the `NodeAttestor "k8s_psat"` block with `NodeAttestor "tpm_devid"` - Add `devid_ca_path` pointing to the DevID CA cert (mounted from a ConfigMap or Secret) - Add `endorsement_ca_path` pointing to the EK CA bundle (same) - Remove the `k8s_psat`-specific RBAC (TokenReview permission) from `server-rbac.yaml`

The server no longer needs to call the Kubernetes TokenReview API — attestation becomes fully TPM-driven with no Kubernetes API dependency.

Phase 5 — SPIRE Agent Config and DaemonSet Update

Update `manifests/spire/agent-config.yaml`: - Replace `NodeAttestor "k8s_psat"` with `NodeAttestor "tpm_devid"` - Set `devid_cert_path`, `devid_priv_path`, `devid_pub_path` to paths under `/etc/spire/`

Update `manifests/spire/agent-daemonset.yaml`: - Add a `hostPath` volume for `/etc/spire` (where provisioning script writes the blobs) - Remove the projected service account token volume (no longer needed for node attestation) - Keep `hostPID: true` — still required for workload attestation

Phase 6 — Deploy and Re-register

1. Apply updated server ConfigMap and roll the StatefulSet
2. Clear existing registration entries from the SPIRE database
3. Apply updated agent ConfigMap and roll the DaemonSet (agents re-attest via TPM)
4. Re-run the registration jobs for experiments 11 and 21
5. Verify: `spire-server entry show` lists correct entries; demo workloads receive SVIDs

Phase 7 — Automate Provisioning in Reinstall Flow

Add DevID provisioning to the node reinstall pipeline so that a freshly reinstalled node automatically gets its TPM DevID material before the SPIRE agent starts. The `reinstall-nodes.sh` script calls `provision-tpm-devid.sh` after the node rejoins k3s, then the CSR is signed and the cert is deployed before Flux reconciles the SPIRE agent.

This closes the operational loop: reinstall a node → it gets a fresh TPM DevID cert → SPIRE agent attests automatically on first start.

Topics Not Asked About That Matter

SVID Rotation for Long-Running Workloads

The experiments use an **init-container pattern**: fetch SVID once at startup, write to an emptyDir, main container uses it. This is fine for demos but won't handle rotation — after ~1 hour the cert expires and the workload has stale credentials.

For production workloads, the options are:

- **spiffe-helper**: a sidecar that watches the Workload API socket for updated SVIDs and rewrites cert files on disk, then sends SIGHUP to the main process.
- **SPIFFE SDK** (Go, Java, Python): native library that holds a live connection to the Workload API and delivers updated SVIDs to the application via callback.
- **Envoy SDS** (Secret Discovery Service): Envoy natively speaks the SPIFFE Workload API; it handles rotation transparently without application changes.

SPIRE HA

The current server is a single StatefulSet replica with SQLite. SQLite cannot be shared across replicas. High availability requires:

1. Switch the datastore to PostgreSQL or MySQL
2. Run multiple server replicas behind a load balancer

With a single server, a server restart means agents can't renew SVIDs during the outage. Workloads continue with cached SVIDs until they expire. Acceptable for a homelab; a consideration for production.

Federation

SPIRE supports federating trust between different SPIFFE trust domains. For example, `ipc.local` and `cloud.example.com` could exchange trust bundles so workloads in each cluster can verify SVIDs from the other. Not needed until there are multiple clusters or external partners, but the mechanism is built in to SPIRE.

The Trust Root and Security Boundary

SPIRE's security model on this cluster roots trust in Kubernetes itself — the PSAT is issued and validated by the Kubernetes API server. Consequently:

- If someone can create pods in the **spire** namespace or compromise the **spire-server** ServiceAccount, they own the CA and can issue arbitrary SVIDs.
- The **spire** namespace is the security boundary, not individual workloads.
- This is the standard SPIRE deployment posture, documented in the experiment README.

Vault Integration via JWT SVID

A natural next step for this cluster (Vault is already deployed): pods can authenticate to Vault using a SPIRE-issued JWT SVID via Vault's JWT auth method. No Vault tokens or secrets need to be injected into pods. The flow is:

1. Pod fetches JWT SVID (audience: `vault`)
2. Pod calls `vault write auth/jwt/login jwt=<svid> role=<role>`
3. Vault validates the JWT against the SPIRE trust bundle (configured as a JWKS endpoint)
4. Vault returns a short-lived Vault token

This replaces Vault's Kubernetes auth method with a stronger, SPIFFE-native identity.

Configuration Reference

Key Files

Path	Purpose
<code>manifests/spire/server-config.yaml</code>	SPIRE Server ConfigMap (trust domain, attestors, storage)
<code>manifests/spire/agent-config.yaml</code>	SPIRE Agent ConfigMap (server address, attestors, socket path)
<code>manifests/spire/server-statefulset.yaml</code>	SPIRE Server StatefulSet + headless service
<code>manifests/spire/agent-daemonset.yaml</code>	SPIRE Agent DaemonSet (hostPID, tolerations, socket hostPath)
<code>manifests/spire/server-rbac.yaml</code>	Server SA, ClusterRole (TokenReview, nodes, pods)
<code>manifests/spire/agent-rbac.yaml</code>	Agent SA, ClusterRole (pods, nodes, nodes/proxy)
<code>clusters/ipc/spire.yaml</code>	Flux Kustomization (path: <code>./manifests/spire</code> , interval: 10m)
<code>experiments/11-spire/</code>	Workload SVID fetch demo
<code>experiments/21-mtls-spire/</code>	mTLS between two workloads

Server Configuration Summary

Setting	Value
Trust domain	<code>ipc.local</code>
Node attesor	<code>k8s_psat</code> , cluster: <code>ipc</code> , allowed SA: <code>spire:spire-agent</code>
Key manager	Disk (<code>/run/spire/data/keys.json</code>)
Datastore	SQLite3 (<code>/run/spire/data/datastore.sqlite3</code>)
Notifier	<code>k8s_bundle</code> → ConfigMap <code>spire-bundle</code>

Setting	Value
gRPC port	8081
Health port	8080 (/live, /ready)

Agent Configuration Summary

Setting	Value
Trust domain	ipc.local
Server	spire-server:8081
Trust bundle	/run/spire/bundle/bundle.crt
Workload API socket	/run/spire/sockets/agent.sock
Node attester	k8s_psat, cluster: ipc
Workload attester	k8s, skip_kubelet_verification: true, use_new_container_locator: true
Key manager	Memory
Health port	8080